Lecture 1: Introduction

Beakal Gizachew Assefa

Motivation

- Why study Parallel Programming?

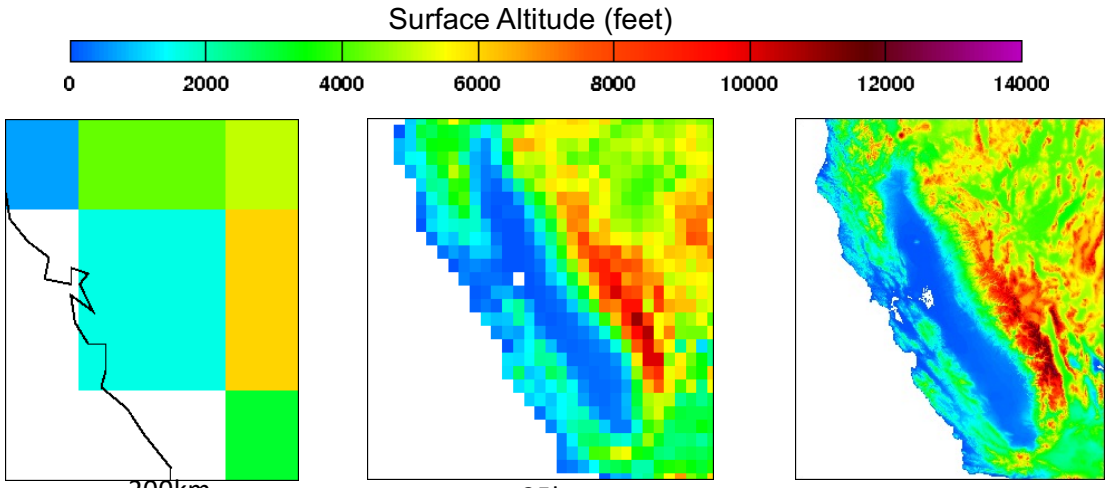
- Why do we need Parallel Computers?

Why do we need parallel computers?

- For higher performance
  - Fast execution
- In fact parallel computing is *OLD*
  - The history goes back to the 1960s
  - Specialized computers (*supercomputers*)
- Scientific and Industrial Applications
  - Requires high performance capability
- Technology Trends
  - Because we have no choice ☺
  - Parallel architectures are everywhere

Scientific and Industrial Problems

- **Climate Change:** Understand the effects of global warming



1 km

Multi--physics

More years/day

50-100 runs

Image Source: Bill Collins

- Many more simulations require high performance computing capability

Other Applications

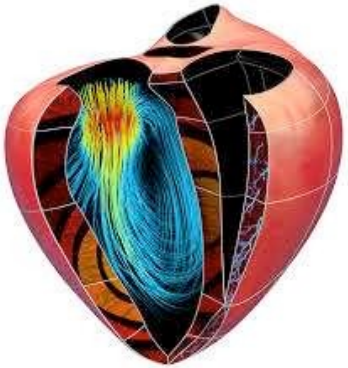
Drug discovery

Energy research

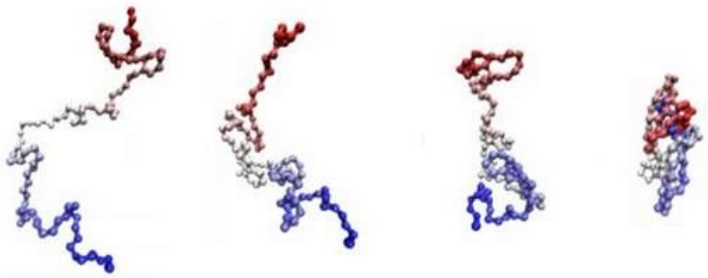
Experiments might be too difficult, expensive, slow, dangerous or impossible

Data Analysis

Cardiac Modeling

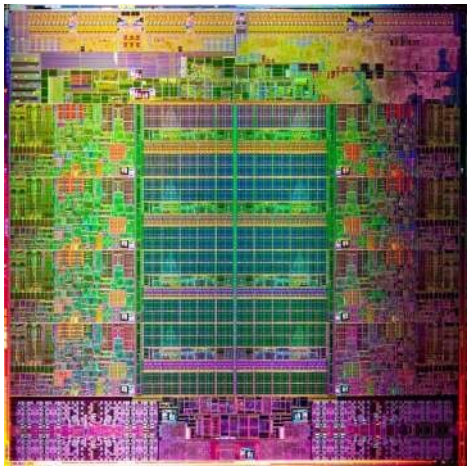


Protein Folding



All Computers are Parallel Computers

Quad-core Samsung Smart TV



Intel Sandy Bridge
8 Cores

(img. Source Anandtech)

Dual-core A8 – iphone 6 chip

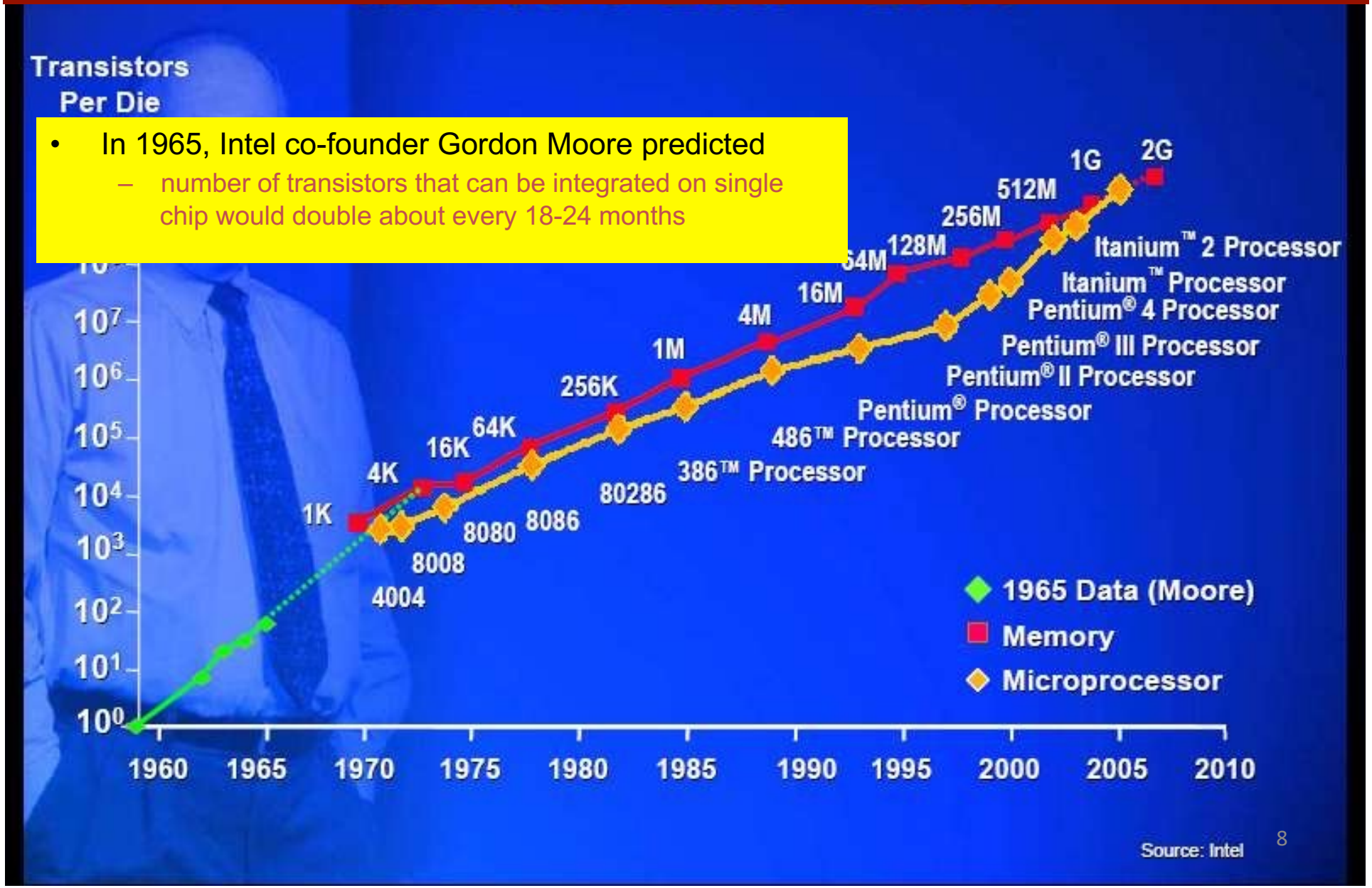
Tianhe-2 in China ~34 Petaflops

1st Trend in Computer Architecture

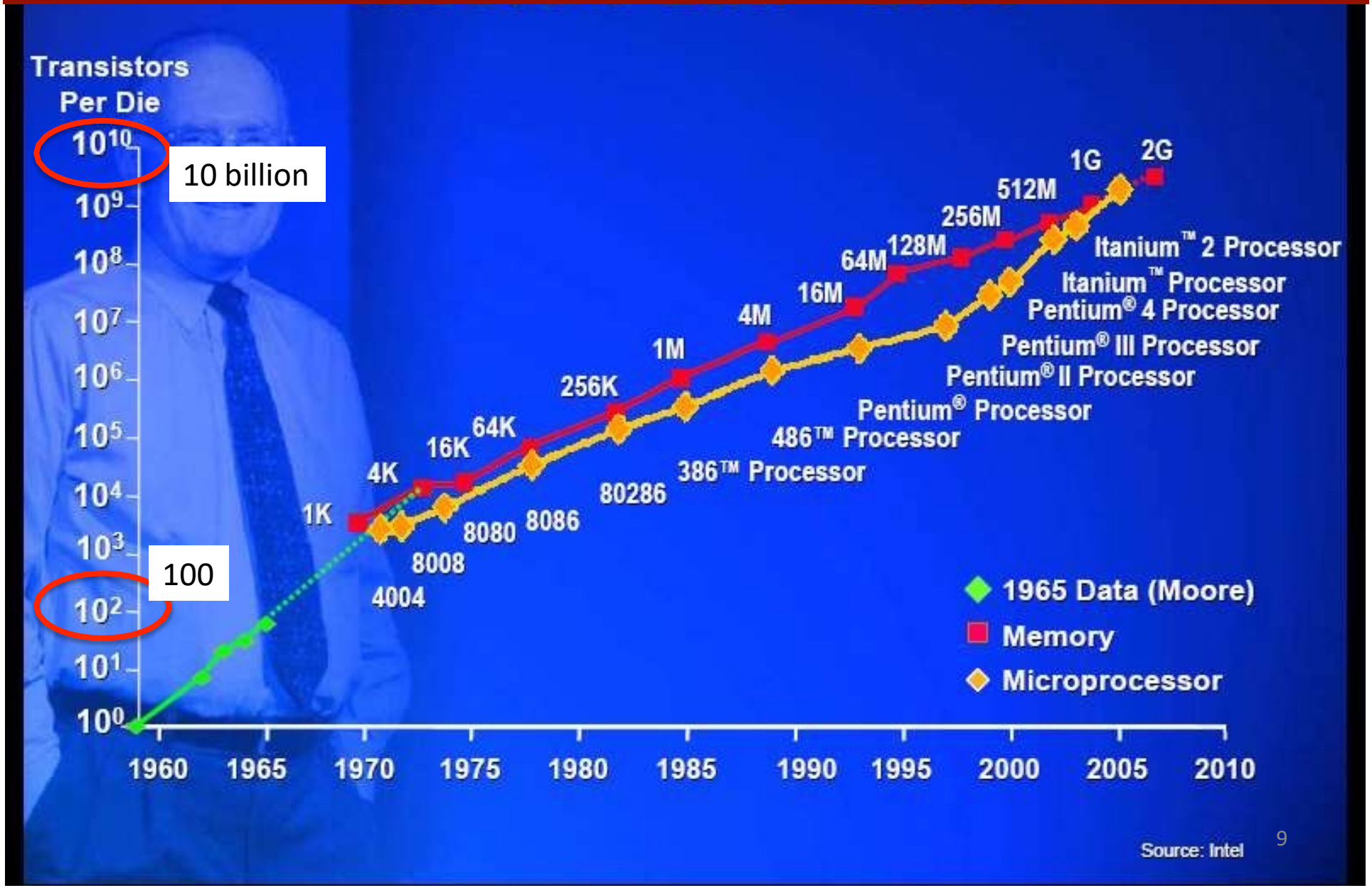
- Massive on-chip parallelism
  - Reinterpretation of Moore's law
  - No clock increases → hundreds of simple "cores" per chip
  - A thousand cores on a chip by 2020
  - 1 Million to 1 Billion--way system level parallelism
  - Already have 64-core chips

Tianhe-2 in China ~34 Petaflops

Moore's Law: 2X transistors / 18–24 months



Moore's Law: 2X transistors / 18–24 months



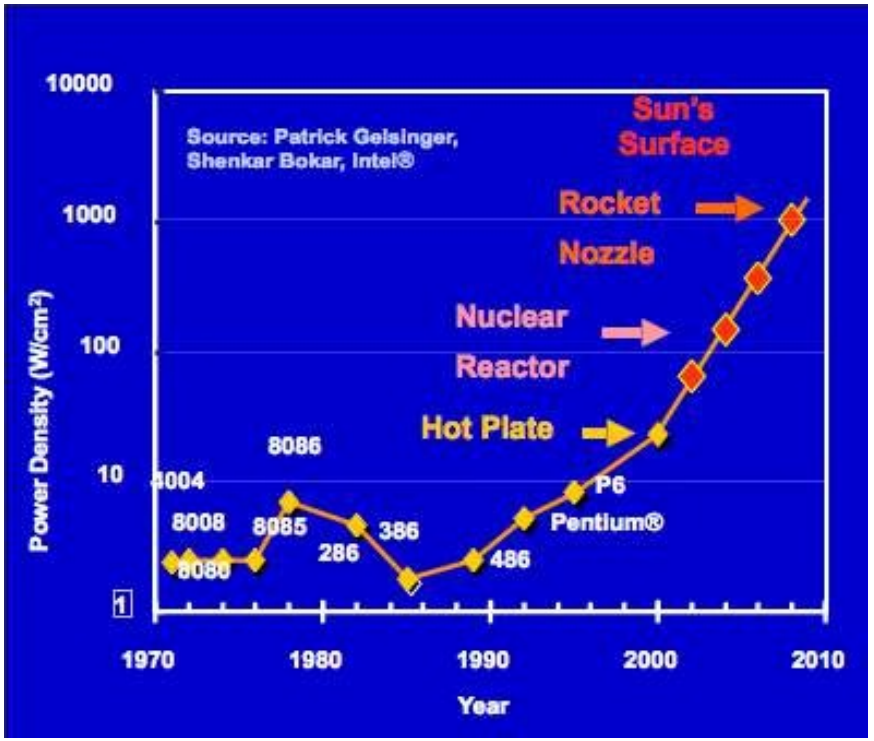
End of Single Processor Era

- Smaller transistors = faster processors.
- Faster processors = increased power consumption.
- Increased power consumption = increased heat.
- Increased heat = unreliable processors.

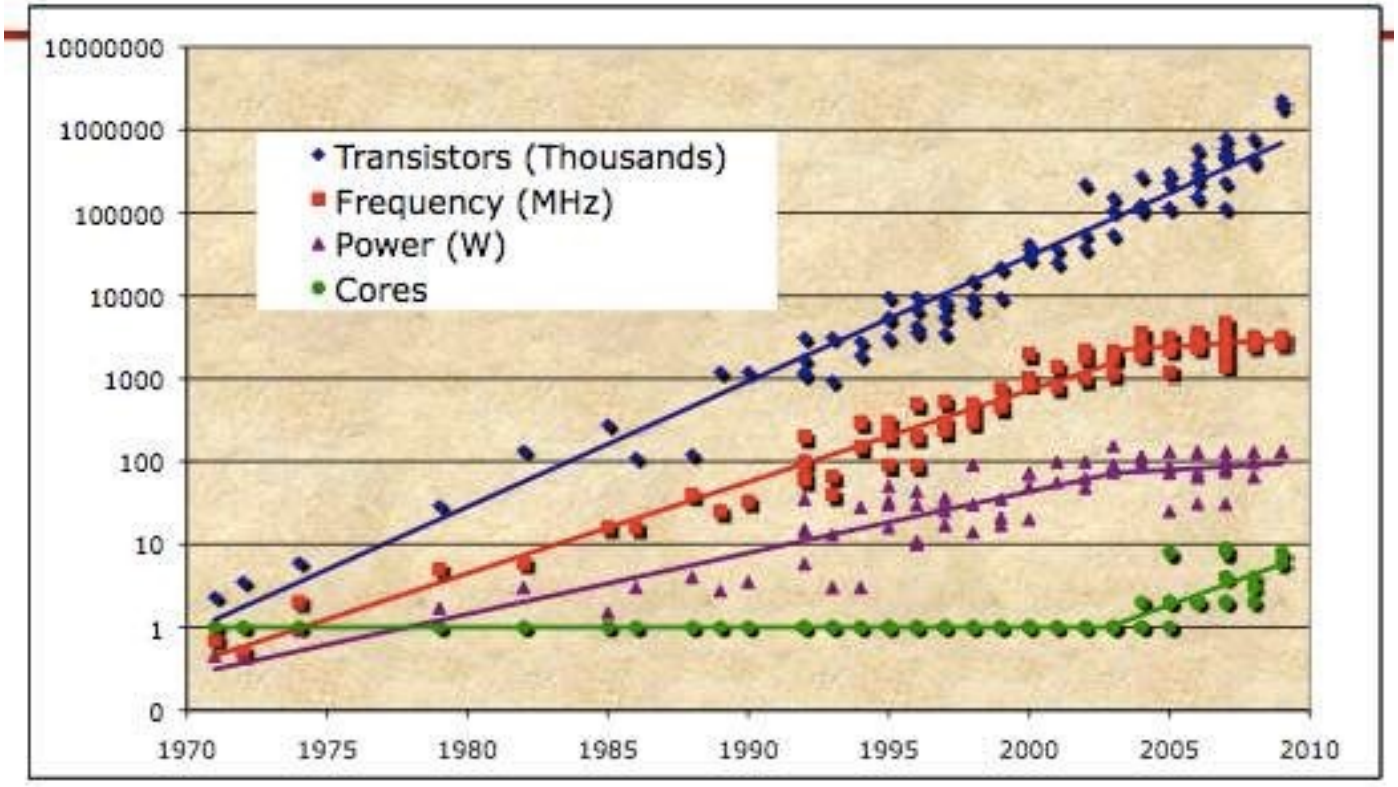
Power limits were reached around 2004.

Solution: Start of multi--core era

Requires parallel programming!



Revolution in Processor

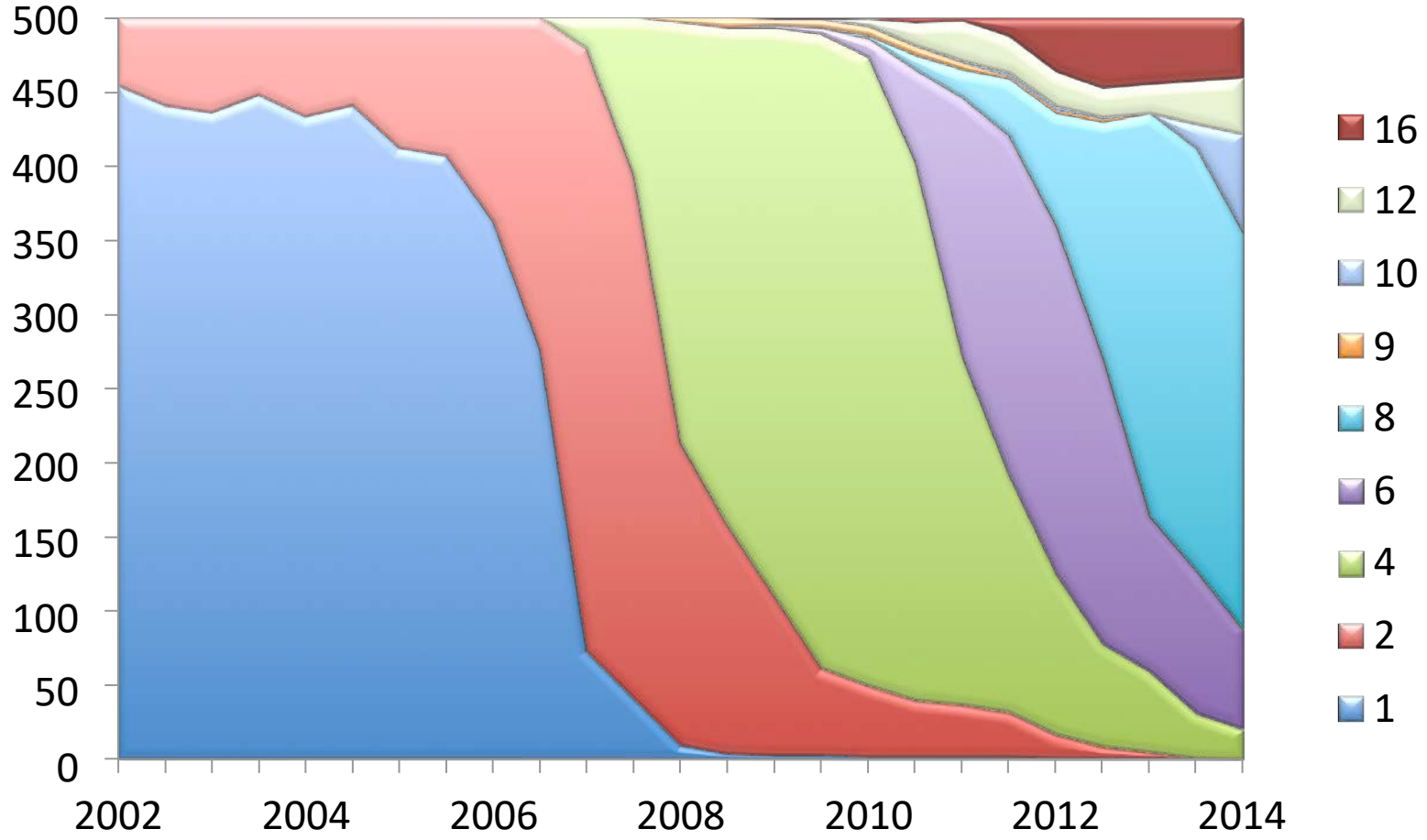


- Chip density is continuing to increase ~2x every 2 years
- Clock speed is not increasing

- # of cores will double instead
- Power is under control (for now)

Source: Kunle Olukotun

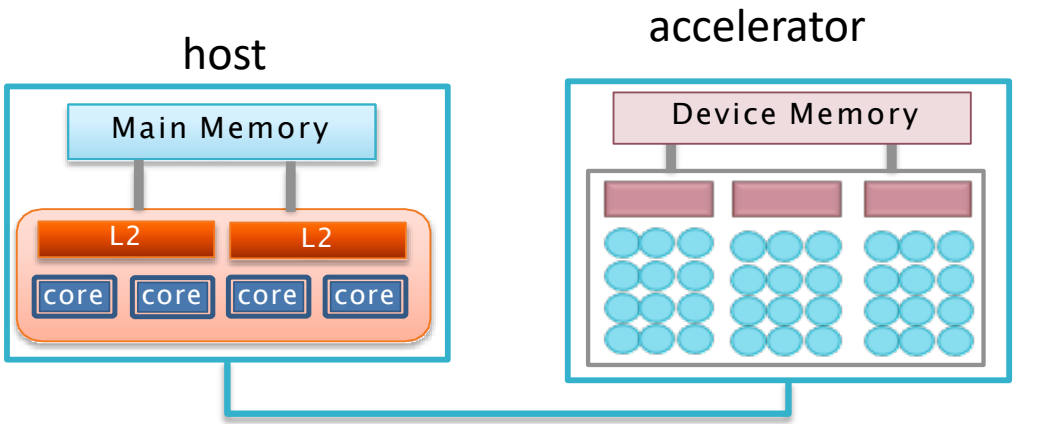
Number of Cores



There are typically 2-4 sockets per node.

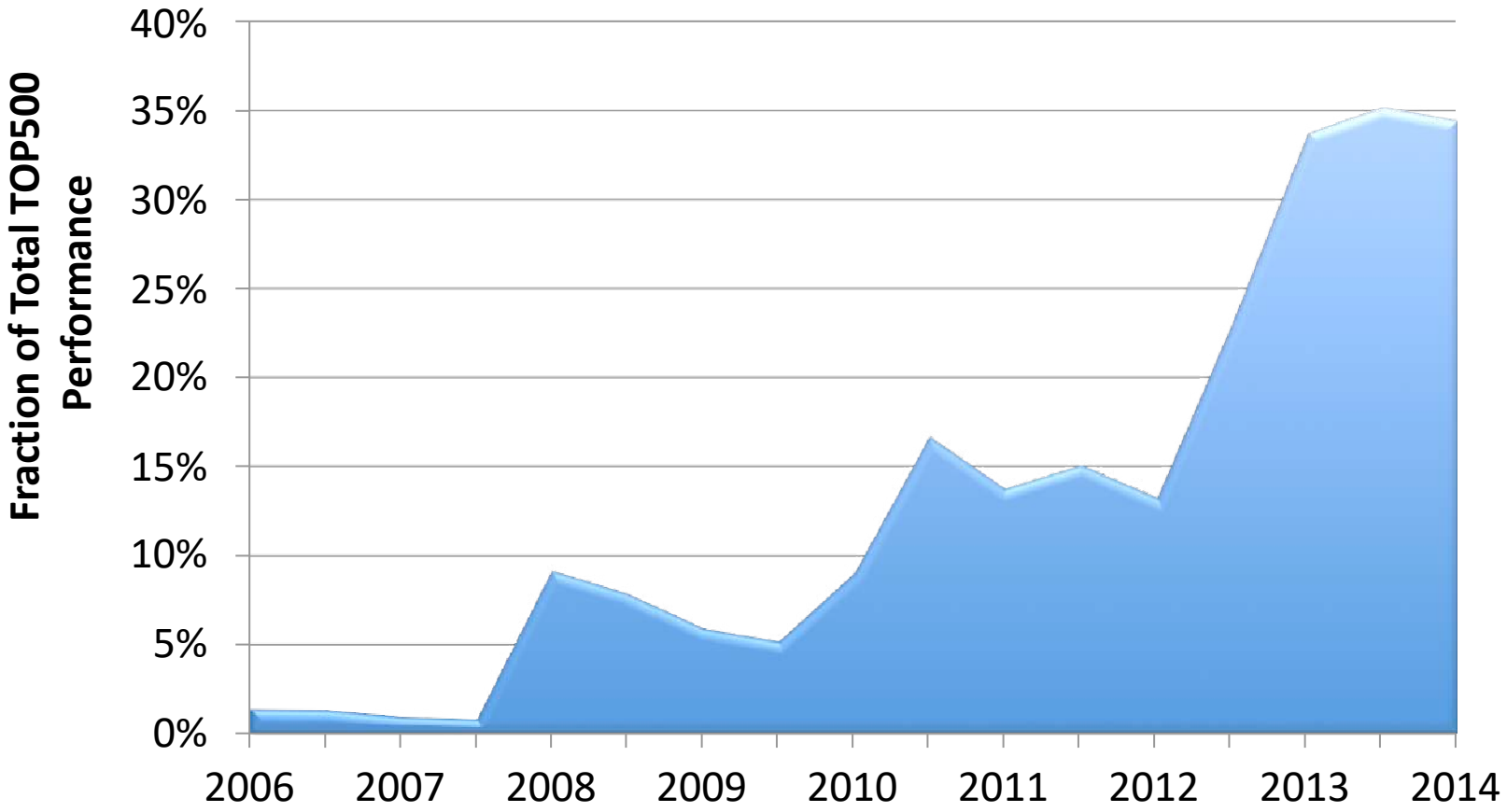
2nd Trend in Computer Architecture

- Heterogeneous systems (host + accelerator)
  - x86 too energy intensive → more lightweight cores

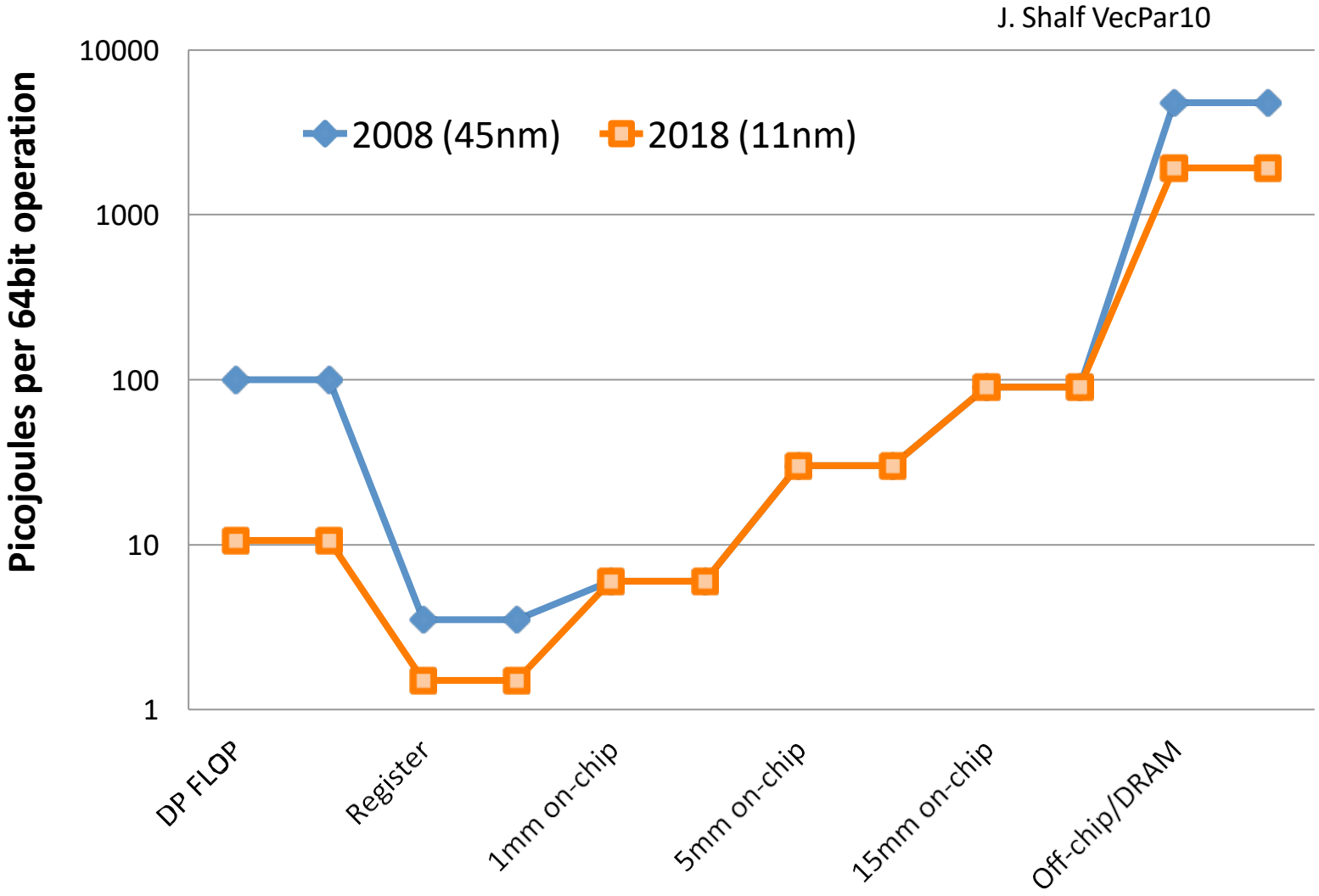


Examples:
Nvidia GPUs
Intel Xeon Phi
AMD APU

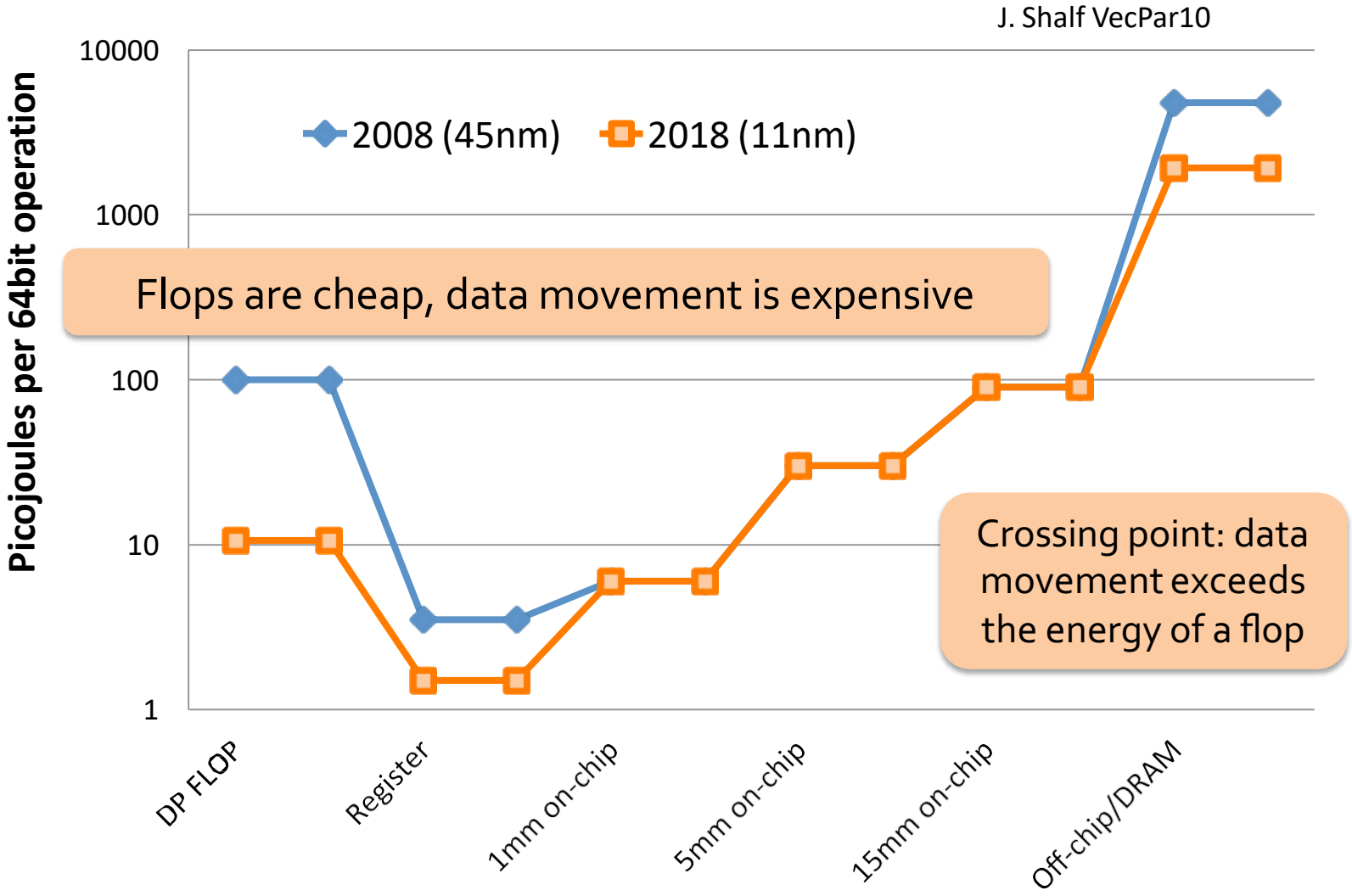
Performance Share of Accelerators in Top 500



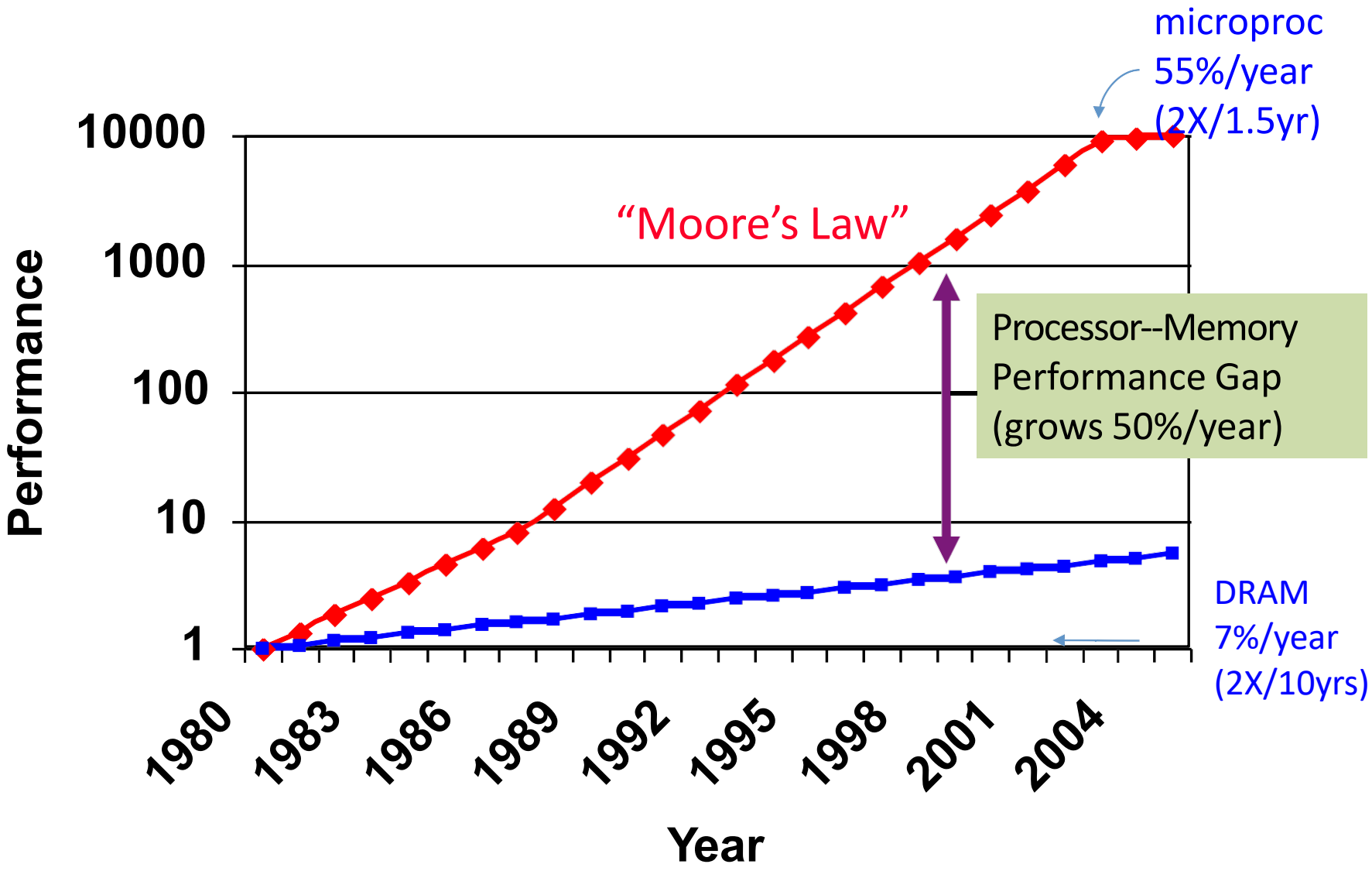
3rd Trend in Computer Architecture



3rd Trend in Computer Architecture



Processor--Memory Performance Gap



Performance

- This course is about ``Performance”
- Performance engineering
  - In this class, we are not only interested in solving a problem in parallel, but also developing a ***high performant code.***

Parallelism | Heterogeneity | Data Locality

Challenges of Parallel Programming

- Serial vs Parallel
  - What kind of problems you may encounter when writing parallel programs?
- Communication
  - Participating tasks need to communicate
- Synchronization
  - Tasks need to coordinate
- Load imbalance
  - Some tasks may do more work than others, need to balance
- Parallelization Overhead
  - Above features lead to overhead

Concurrent, Distributed vs Parallel

- **Concurrent:** a program with multiple tasks *in progress* at any instant
  - OS executing multiple tasks on a single CPU
- **Distributed:** a program which may be cooperating with other programs to solve a problem
  - Loosely coupled, geographically far away
  - Independently created programs
  - Cloud computing, web search, bank transactions etc.
- **Parallel:** a program with multiple tasks *cooperating closely* to solve a problem
  - Fast network connection, tightly coupled system
  - Single application or program

Course Basics

- Website
  - Google classroom
  - All course materials will be posted (hopefully before the class)
- Main Book
  - An Introduction to Parallel Programming by Peter Pacheco (ISBN: 978–0–12–-374260–5)

- *Additional reading materials will be posted on as needed*

Requirements for the Class

- Basic knowledge in Computer Architecture
  - Memory, cache, core concepts
- **C Programming**
  - Pointers, dynamic memory allocation, multidim arrays, std input/output
    - http://www.cprogramming.com/tutorial/c--tutorial.html
- Basic Linux commands
  - Remote access, transfer files to/from clusters
  - Develop and test programs on clusters
- Command line editors
  - Highly recommend getting familiar with Emacs or Vi
- Want to locally develop your programs?
  - Have to install parallel libraries locally
  - Still need to collect performance data on a cluster